FENI UNIVERSITY

Department of Computer Science and Engineering (CSE)

Program: Undergraduate Bachelor of Science (B.Sc. in CSE)

LAB PROJECT REPORT

Course Title: Database Management System Lab

PROJECT TITLE

Inventory / Product Management System

Submitted By	Submitted To
Name: Rejowan Ferdous Khan **Student ID:** 251031024 **Batch:** CSE 34th Batch **Program:** B.Sc. in CSE **Department:** Dept. of CSE, Feni University	**Name:** Faria Sultana Bintha Rohman **Designation:** Lab Technical Officer (LTO) **Department:** B.Sc. in CSE, Feni University

Date of Submission: November 05, 2026

1. Project Objective

The objective of this lab project is to design and develop a single-page web application connected to a relational database for managing store inventory records. The system demonstrates practical implementation of core SQL operations (`CREATE` , `INSERT` , `SELECT` , `WHERE` , `UPDATE` , `DELETE`) and SQL aggregate functions (`COUNT()` , `SUM()` , `AVG()`) through a clean user interface. The dataset used in this project represents real agricultural and livestock equipment from Farm Solution BD.

Key Features Implemented

1. **Add Records:** Users can add new product categories and save new product records directly from the single-page interface.
2. **View Records:** Saved inventory items are displayed in a structured table with sequential product IDs, categories, stock quantities, prices in Bangladeshi Taka (`৳`), and manager names.
3. **Search and Filter:** Users can search products by ID, name, category, or manager name, and filter records by category using SQL `WHERE` and `LIKE` clauses.
4. **Update Records:** Existing product details can be edited and updated using a modal dialog form.
5. **Delete Records:** Unused categories and product records can be deleted with foreign-key protection and confirmation prompts.
6. **Summary Dashboard:** Displays total products (`COUNT`), total stock units (`SUM`), average unit price (`AVG`), and total stock valuation (`SUM`) formatted in Bangladeshi Taka (Lakh/Crore grouping).

2. Tools Used & Project File Structure

2.1 Technology Stack

Component	Technology	Description
Frontend	HTML5, CSS3, JavaScript (`index.html` , `style.css` , `script.js`)	Single-page user interface, forms, inventory table, search/filter bar, and modal dialogs.
Backend	Node.js, Express (`server.js`)	REST API server handling HTTP requests, input validation, and SQL execution.
Database Driver	`mysql2`	Connects Node.js with the MySQL database using parameterized SQL queries.
Database	MySQL (`inventory_db`)	Relational database storing the `categories` and `products` tables.

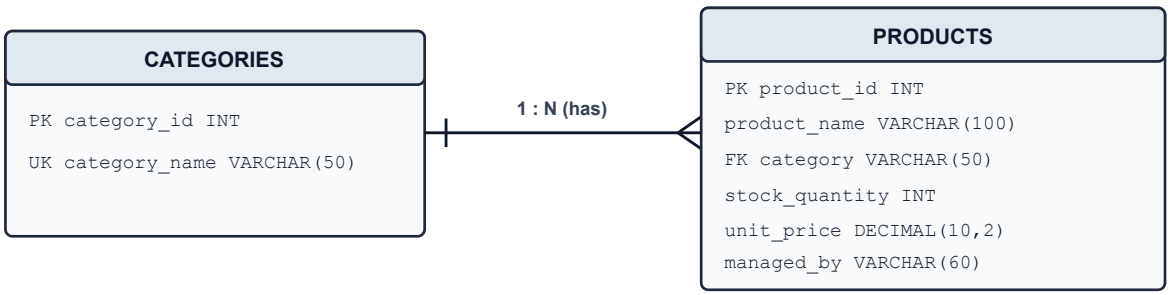
2.2 Project File Structure

```
Inventory-Product-Management-System/
├── database.sql
├── server.js
├── package.json
├── LAB_REPORT.md
├── public/
│   ├── index.html
│   ├── style.css
│   └── script.js
└── screenshots/
    ├── dashboard-overview.png
    └── edit-record-modal.png
```

3. Database Table Design & ER Diagram

3.1 Entity-Relationship (ER) Diagram

The database `inventory_db` contains two related tables: `categories` (parent table) and `products` (child table). One category can be associated with multiple products (`1:N` relationship), enforced by a `FOREIGN KEY` constraint on `products.category` referencing `categories(category_name)` .



3.2 Table Structure

Table	Column Name	Data Type	Constraints	Description
`categories`	`category_id`	`INT`	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique ID for each category
`categories`	`category_name`	`VARCHAR(50)`	`NOT NULL`, `UNIQUE`	Name of the product category
`products`	`product_id`	`INT`	`PRIMARY KEY`, `AUTO_INCREMENT`	Sequential ID for each product
`products`	`product_name`	`VARCHAR(100)`	`NOT NULL`	Name of the product
`products`	`category`	`VARCHAR(50)`	`NOT NULL`, `FOREIGN KEY`	References `categories(category_name)`
`products`	`stock_quantity`	`INT`	`NOT NULL`, `CHECK (>= 0)`	Available units in stock
`products`	`unit_price`	`DECIMAL(10, 2)`	`NOT NULL`, `CHECK (>= 0.00)`	Unit price in Bangladeshi Taka (`৳`)
`products`	`managed_by`	`VARCHAR(60)`	`NOT NULL`	Store manager responsible for the item

4. Key SQL Queries

4.1 Database and Table Creation (`CREATE`)

```
CREATE DATABASE IF NOT EXISTS inventory_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE inventory_db;

CREATE TABLE IF NOT EXISTS categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
    unit_price DECIMAL(10, 2) NOT NULL CHECK (unit_price >= 0.00),
    managed_by VARCHAR(60) NOT NULL,
    CONSTRAINT fk_products_category
        FOREIGN KEY (category) REFERENCES categories(category_name)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);
```

4.2 Data Insertion (`INSERT`)

```
INSERT INTO categories (category_name) VALUES
('Cow farm tools'),
('Goat farm tools');

INSERT INTO products (product_name, category, stock_quantity, unit_price, managed_by) VALUES
('Cow Rubber Ear Tag', 'Cow farm tools', 10000, 50.00, 'Rejowan'),
('Cow Rubber Neck Tag', 'Cow farm tools', 10000, 45.00, 'Tanjil'),
('Goat Rubber Ear Tag', 'Goat farm tools', 10000, 40.00, 'Rejowan'),
('Goat Neck Plastic Tag', 'Goat farm tools', 10000, 30.00, 'Tanjil'),
('Cow Neck Plastic Tag', 'Cow farm tools', 10000, 40.00, 'Rejowan'),
('Ear Tag Applicator', 'Cow farm tools', 70, 1550.00, 'Tanjil'),
('Heavy Duty Nylon Cow Collar Belt', 'Cow farm tools', 200, 700.00, 'Rejowan'),
('Automatic Vaccine Gun', 'Cow farm tools', 100, 2500.00, 'Tanjil'),
('Cattle Weight Measuring Tape', 'Cow farm tools', 150, 450.00, 'Rejowan'),
('2.5 Liter Calf Feeder', 'Cow farm tools', 80, 450.00, 'Tanjil'),
('Spring Cattle Nose Ring', 'Cow farm tools', 250, 750.00, 'Rejowan'),
('Trocar Cannula', 'Goat farm tools', 350, 45.00, 'Tanjil'),
('Automatic Cattle Water Bowl', 'Cow farm tools', 75, 1650.00, 'Rejowan'),
('Dairy Farm Management Book', 'Cow farm tools', 45, 600.00, 'Tanjil'),
('Nylon Goat Neck Collar Belt', 'Goat farm tools', 300, 350.00, 'Rejowan');
```

4.3 View Records and Categories (`SELECT`)

```
SELECT c.category_id, c.category_name, COUNT(p.product_id) AS product_count
FROM categories c
LEFT JOIN products p ON p.category = c.category_name
GROUP BY c.category_id, c.category_name
ORDER BY c.category_name ASC;

SELECT product_id, product_name, category, stock_quantity, unit_price, managed_by
FROM products
ORDER BY product_id ASC;
```

4.4 Search and Filter Records (`SELECT` with `WHERE` and `LIKE`)

```
SELECT product_id, product_name, category, stock_quantity, unit_price, managed_by
FROM products
WHERE (product_id = 1 OR product_name LIKE '%Tag%' OR category LIKE '%Tag%' OR managed_by LIKE
'%Tag%')
  AND category = 'Cow farm tools'
ORDER BY product_id ASC;
```

4.5 Update Record (`UPDATE` with `WHERE`)

```
UPDATE products
SET product_name = 'Cow Rubber Ear Tag',
    category = 'Cow farm tools',
    stock_quantity = 10000,
    unit_price = 50.00,
    managed_by = 'Rejowan'
WHERE product_id = 1;
```

4.6 Delete Record (`DELETE` with `WHERE`)

```
DELETE FROM products
WHERE product_id = 15;
```

```
DELETE FROM categories
WHERE category_id = 2;
```

4.7 Summary Queries Using Aggregate Functions (`COUNT` , `SUM` , `AVG`)

```
SELECT
    COUNT(*) AS total_products,
    COALESCE(SUM(stock_quantity), 0) AS total_stock,
    COALESCE(ROUND(AVG(unit_price), 2), 0) AS average_price,
    COALESCE(ROUND(SUM(stock_quantity * unit_price), 2), 0) AS total_inventory_value
FROM products;
```

Total Products (`COUNT(*)`)	Total Stock Units (`SUM(stock_quantity)`)	Average Unit Price (`AVG(unit_price)`)	Total Stock Valuation (`SUM(qty * price)`)
15	**51,620 units**	**৳616.67**	**৳31,11,000.00**

5. Screenshots of the Working Application

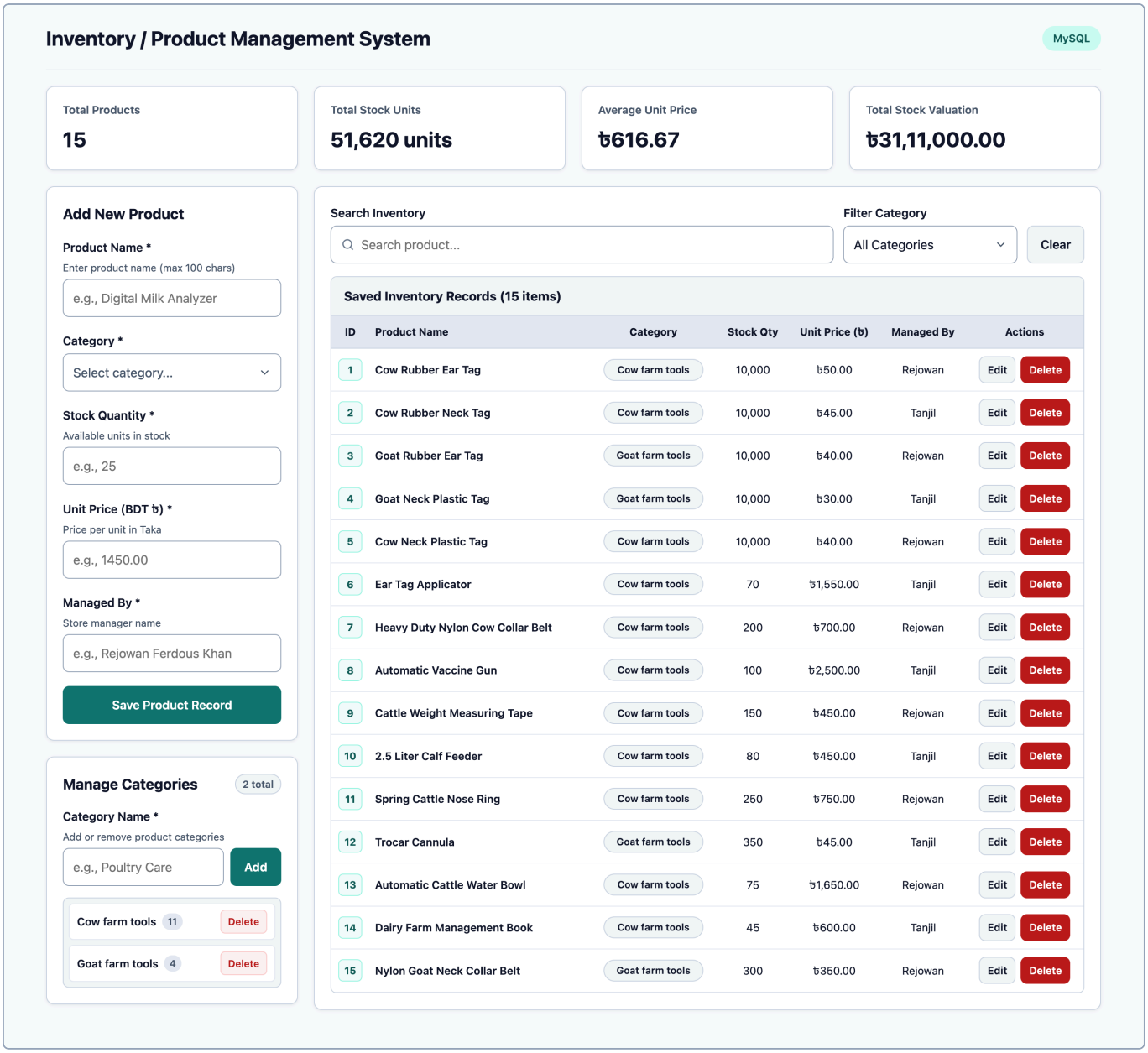


Figure 1: Main Single-Page Inventory Management Interface (15 Real Products)

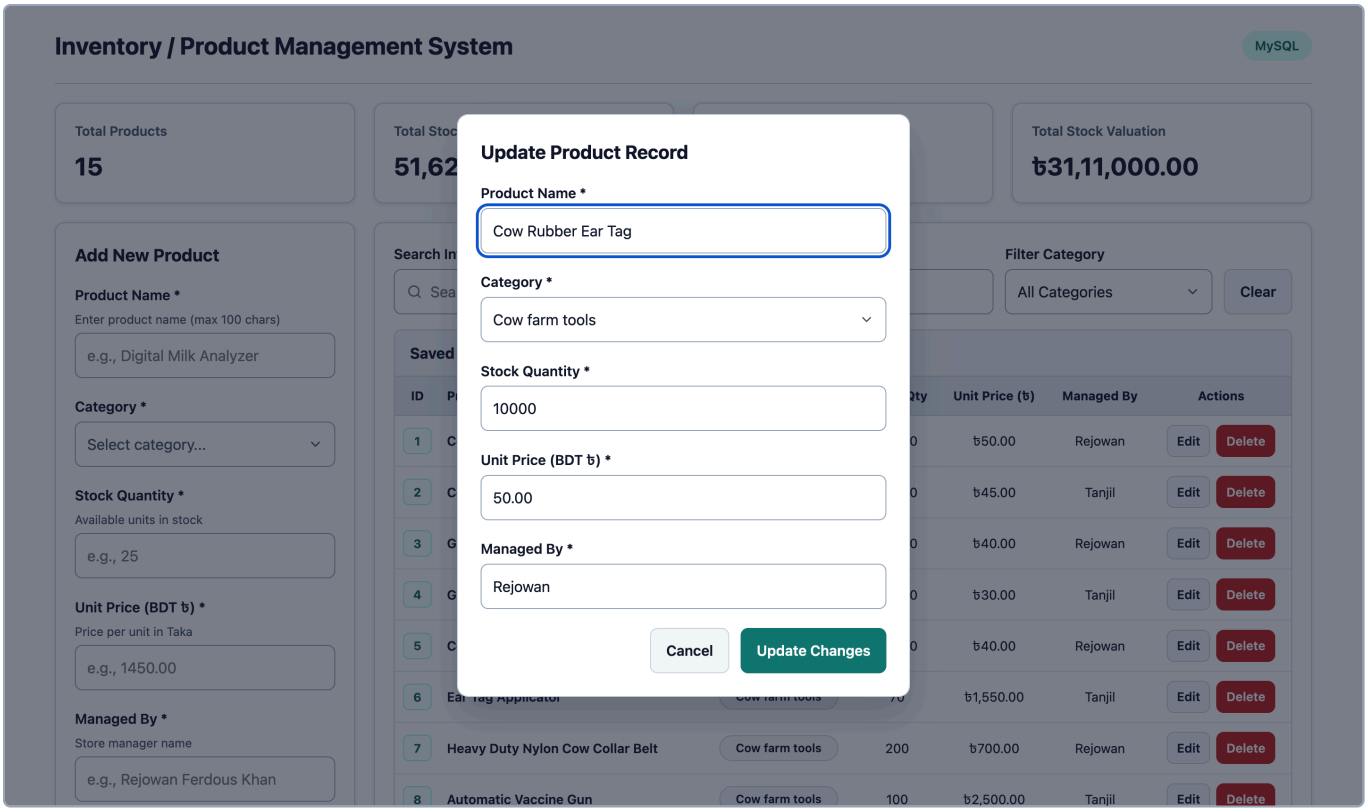


Figure 2: Update Product Record Modal Dialog

6. How to Run the Project

1. **GitHub Repository:** `https://github.com/RUMON07/Inventory-Product-Management-System`
2. **Database Setup:** Start the local MySQL server (such as XAMPP on `localhost:3306` with user `root`) and import `database.sql`, or simply start the Node.js server which initializes the schema and records from `database.sql` automatically.
3. **Install Dependencies and Start Server:**

```
npm install
npm start
```

4. **Open in Browser:** Visit `http://localhost:3000` in any web browser.

7. Complete Source Code (Latest Version)

7.1 Database Script (`database.sql`)

```
CREATE DATABASE IF NOT EXISTS inventory_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE inventory_db;

CREATE TABLE IF NOT EXISTS categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
    unit_price DECIMAL(10, 2) NOT NULL CHECK (unit_price >= 0.00),
    managed_by VARCHAR(60) NOT NULL,
    CONSTRAINT fk_products_category
        FOREIGN KEY (category) REFERENCES categories(category_name)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE products;
TRUNCATE TABLE categories;
SET FOREIGN_KEY_CHECKS = 1;

INSERT INTO categories (category_name) VALUES
('Cow farm tools'),
('Goat farm tools');

INSERT INTO products (product_name, category, stock_quantity, unit_price, managed_by) VALUES
('Cow Rubber Ear Tag', 'Cow farm tools', 10000, 50.00, 'Rejowan'),
('Cow Rubber Neck Tag', 'Cow farm tools', 10000, 45.00, 'Tanjil'),
('Goat Rubber Ear Tag', 'Goat farm tools', 10000, 40.00, 'Rejowan'),
('Goat Neck Plastic Tag', 'Goat farm tools', 10000, 30.00, 'Tanjil'),
('Cow Neck Plastic Tag', 'Cow farm tools', 10000, 40.00, 'Rejowan'),
('Ear Tag Applicator', 'Cow farm tools', 70, 1550.00, 'Tanjil'),
('Heavy Duty Nylon Cow Collar Belt', 'Cow farm tools', 200, 700.00, 'Rejowan'),
('Automatic Vaccine Gun', 'Cow farm tools', 100, 2500.00, 'Tanjil'),
('Cattle Weight Measuring Tape', 'Cow farm tools', 150, 450.00, 'Rejowan'),
('2.5 Liter Calf Feeder', 'Cow farm tools', 80, 450.00, 'Tanjil'),
('Spring Cattle Nose Ring', 'Cow farm tools', 250, 750.00, 'Rejowan'),
('Trocar Cannula', 'Goat farm tools', 350, 45.00, 'Tanjil'),
('Automatic Cattle Water Bowl', 'Cow farm tools', 75, 1650.00, 'Rejowan'),
('Dairy Farm Management Book', 'Cow farm tools', 45, 600.00, 'Tanjil'),
('Nylon Goat Neck Collar Belt', 'Goat farm tools', 300, 350.00, 'Rejowan');
```

7.2 Package Configuration (`package.json`)

```
{
  "name": "inventory-management-system",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "db:seed": "node server.js --seed"
```

```
  },
  "dependencies": {
    "express": "^5.1.0",
    "mysql2": "^3.14.0"
  }
}
```

7.3 Backend Server (`server.js`)

```
const path = require('path');
const fs = require('fs');
const os = require('os');
const express = require('express');
const mysql = require('mysql2/promise');

const dbConfig = {
  host: process.env.DB_HOST || 'localhost',
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_NAME || 'inventory_db',
};

function validateProduct(body = {}) {
  const product_name = String(body.product_name ?? '').trim();
  const category = String(body.category ?? '').trim();
  const managed_by = String(body.managed_by ?? '').trim();
  const rawStock = body.stock_quantity;
  const rawPrice = body.unit_price;

  if (!product_name || product_name.length > 100) {
    return { error: 'Product name is required (max 100 characters).' };
  }
  if (!category || category.length > 50) {
    return { error: 'Category is required (max 50 characters).' };
  }
  if (!managed_by || managed_by.length > 60) {
    return { error: 'Manager name is required (max 60 characters).' };
  }
  if (rawStock === '' || rawStock === null || rawStock === undefined) {
    return { error: 'Stock quantity is required.' };
  }
  const stock_quantity = Number(rawStock);
  if (!Number.isInteger(stock_quantity) || stock_quantity < 0) {
    return { error: 'Stock quantity must be a non-negative whole number.' };
  }
  if (rawPrice === '' || rawPrice === null || rawPrice === undefined) {
    return { error: 'Unit price is required.' };
  }
  const parsedPrice = Number(rawPrice);
  if (!Number.isFinite(parsedPrice) || parsedPrice < 0) {
    return { error: 'Unit price must be 0.00 or higher.' };
  }
  const unit_price = Number(parsedPrice.toFixed(2));

  return { value: { product_name, category, stock_quantity, unit_price, managed_by } };
}

function buildFilterClause({ search = '', category = '' } = {}) {
  const conditions = [];
  const params = [];
  const trimmedSearch = String(search).trim();
  const trimmedCategory = String(category).trim();

  if (trimmedSearch) {
    const idCandidate = Number(trimmedSearch.replace(/^#/, ''));
```

```
    const pattern = `%${trimmedSearch}%`;
    if (Number.isInteger(idCandidate) && idCandidate > 0) {
      conditions.push('(product_id = ? OR product_name LIKE ? OR category LIKE ? OR managed_by LIKE
?)');
      params.push(idCandidate, pattern, pattern, pattern);
    } else {
      conditions.push('(product_name LIKE ? OR category LIKE ? OR managed_by LIKE ?)');
      params.push(pattern, pattern, pattern);
    }
  }
  if (trimmedCategory) {
    conditions.push('category = ?');
    params.push(trimmedCategory);
  }

  return {
    whereSql: conditions.length ? ` WHERE ${conditions.join(' AND ')}` : '',
    params,
  };
}

function parseId(rawId) {
  const id = Number(rawId);
  return Number.isInteger(id) && id > 0 ? id : null;
}

function validateCategory(body = {}) {
  const category_name = String(body.category_name ?? '')
    .replace(/\s+/g, ' ')
    .trim();
  if (!category_name || category_name.length > 50) {
    return { error: 'Category name is required (max 50 characters).' };
  }
  return { value: category_name };
}

function escapeSqlString(str) {
  return String(str).replace(/\\/g, '\\\\').replace(/'/g, "''");
}

async function resequenceCategories(pool) {
  const [rows] = await pool.execute('SELECT category_id FROM categories ORDER BY category_id ASC');
  for (let i = 0; i < rows.length; i += 1) {
    const currentId = Number(rows[i].category_id);
    const targetId = i + 1;
    if (currentId !== targetId) {
      await pool.execute('UPDATE categories SET category_id = ? WHERE category_id = ?', [
        targetId,
        currentId,
      ]);
    }
  }
  return rows.length;
}

async function resequenceProducts(pool) {
  const [rows] = await pool.execute('SELECT product_id FROM products ORDER BY product_id ASC');
  for (let i = 0; i < rows.length; i += 1) {
    const currentId = Number(rows[i].product_id);
    const targetId = i + 1;
    if (currentId !== targetId) {
      await pool.execute('UPDATE products SET product_id = ? WHERE product_id = ?', [
        targetId,
        currentId,
      ]);
    }
  }
  return rows.length;
```

```
}

async function syncDatabaseSqlFile(pool) {
  const [catRows] = await pool.execute(
    'SELECT category_name FROM categories ORDER BY category_name ASC'
  );
  const [prodRows] = await pool.execute(
    'SELECT product_name, category, stock_quantity, unit_price, managed_by FROM products ORDER BY
product_id ASC'
  );

  const lines = [
    'CREATE DATABASE IF NOT EXISTS inventory_db',
    '    CHARACTER SET utf8mb4',
    '    COLLATE utf8mb4_unicode_ci;',
    '',
    'USE inventory_db;',
    '',
    'CREATE TABLE IF NOT EXISTS categories (',
    '    category_id INT AUTO_INCREMENT PRIMARY KEY,',
    '    category_name VARCHAR(50) NOT NULL UNIQUE',
    ');',
    '',
    'CREATE TABLE IF NOT EXISTS products (',
    '    product_id INT AUTO_INCREMENT PRIMARY KEY,',
    '    product_name VARCHAR(100) NOT NULL,',
    '    category VARCHAR(50) NOT NULL,',
    '    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),',
    '    unit_price DECIMAL(10, 2) NOT NULL CHECK (unit_price >= 0.00),',
    '    managed_by VARCHAR(60) NOT NULL,',
    '    CONSTRAINT fk_products_category',
    '        FOREIGN KEY (category) REFERENCES categories(category_name)',
    '        ON UPDATE CASCADE',
    '        ON DELETE RESTRICT',
    ');',
    '',
    'SET FOREIGN_KEY_CHECKS = 0;',
    'TRUNCATE TABLE products;',
    'TRUNCATE TABLE categories;',
    'SET FOREIGN_KEY_CHECKS = 1;',
  ];

  if (catRows.length > 0) {
    lines.push('');
    lines.push('INSERT INTO categories (category_name) VALUES');
    catRows.forEach((row, idx) => {
      const suffix = idx === catRows.length - 1 ? ';' : ',';
      lines.push(`('${escapeSqlString(row.category_name)}')${suffix}`);
    });
  }

  if (prodRows.length > 0) {
    lines.push('');
    lines.push(
      'INSERT INTO products (product_name, category, stock_quantity, unit_price, managed_by) VALUES'
    );
    prodRows.forEach((row, idx) => {
      const suffix = idx === prodRows.length - 1 ? ';' : ',';
      const price = Number(row.unit_price).toFixed(2);
      lines.push(
        `('${escapeSqlString(row.product_name)}', '${escapeSqlString(row.category)}',
${Number(row.stock_quantity)}, ${price}, '${escapeSqlString(row.managed_by)}')${suffix}`
      );
    });
  }

  lines.push('');
  fs.writeFileSync(path.join(__dirname, 'database.sql'), lines.join('\n'), 'utf8');
```

```
}

function createApp(pool, options = {}) {
  const { syncSqlFile = false } = options;
  const app = express();
  app.disable('x-powered-by');
  app.use((_req, res, next) => {
    res.setHeader('X-Content-Type-Options', 'nosniff');
    res.setHeader('X-Frame-Options', 'DENY');
    res.setHeader(
      'Content-Security-Policy',
      "default-src 'self'; img-src 'self' data:; style-src 'self' 'unsafe-inline'; script-src 'self'
'unsafe-inline'"
    );
    next();
  });
  app.use(express.json());

  app.get(['/favicon.ico', '/favicon.svg'], (_req, res) => {
    res.setHeader('Cache-Control', 'no-store');
    res.setHeader('Content-Type', 'image/svg+xml');
    res.send('<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 16 16"><rect width="16"
height="16" rx="3" fill="#0f766e"/></svg>');
  });

  app.use(express.static(path.join(__dirname, 'public')));

  async function resolveCategoryName(category) {
    const [rows] = await pool.execute(
      'SELECT category_name FROM categories WHERE LOWER(category_name) = LOWER(?)',
      [category]
    );
    return rows.length > 0 ? rows[0].category_name : null;
  }

  app.get('/api/categories', async (_req, res) => {
    const sql = `
      SELECT
        c.category_id,
        c.category_name,
        COUNT(p.product_id) AS product_count
      FROM categories c
      LEFT JOIN products p ON p.category = c.category_name
      GROUP BY c.category_id, c.category_name
      ORDER BY c.category_name ASC
    `;
    const [rows] = await pool.execute(sql);
    res.json(
      rows.map((r) => ({
        category_id: Number(r.category_id),
        category_name: r.category_name,
        product_count: Number(r.product_count || 0),
      }))
    );
  });

  app.post('/api/categories', async (req, res) => {
    const { error, value: category_name } = validateCategory(req.body);
    if (error) return res.status(400).json({ error });

    const [existing] = await pool.execute(
      'SELECT category_id, category_name FROM categories WHERE LOWER(category_name) = LOWER(?)',
      [category_name]
    );
    if (existing.length > 0) {
      return res.status(409).json({
        error: `Category "${existing[0].category_name}" already exists.`,
      });
```

```
  }

  try {
    const currentCount = await resequenceCategories(pool);
    const nextId = currentCount + 1;
    await pool.execute(
      'INSERT INTO categories (category_id, category_name) VALUES (?, ?)',
      [nextId, category_name]
    );
    if (syncSqlFile) await syncDatabaseSqlFile(pool);
    res.status(201).json({ category_id: nextId, category_name, product_count: 0 });
  } catch (err) {
    if (err && (err.code === 'ER_DUP_ENTRY' || /unique/i.test(err.message || ''))) {
      return res.status(409).json({ error: `Category "${category_name}" already exists.` });
    }
    throw err;
  }
});

app.delete('/api/categories/:id', async (req, res) => {
  const id = parseId(req.params.id);
  if (!id) return res.status(400).json({ error: 'Invalid category ID.' });

  const [cats] = await pool.execute(
    'SELECT category_id, category_name FROM categories WHERE category_id = ?',
    [id]
  );
  if (cats.length === 0) {
    return res.status(404).json({ error: 'Category not found.' });
  }
  const { category_name } = cats[0];

  const [usage] = await pool.execute(
    'SELECT COUNT(*) AS cnt FROM products WHERE category = ?',
    [category_name]
  );
  const inUseCount = Number(usage[0]?.cnt || 0);
  if (inUseCount > 0) {
    return res.status(409).json({
      error: `Cannot delete "${category_name}" because ${inUseCount} product(s) still use it.`,
    });
  }

  await pool.execute('DELETE FROM categories WHERE category_id = ?', [id]);
  await resequenceCategories(pool);
  if (syncSqlFile) await syncDatabaseSqlFile(pool);
  res.json({ message: 'Category deleted successfully.', category_id: id, category_name });
});

app.get('/api/products/summary', async (req, res) => {
  const { whereSql, params } = buildFilterClause(req.query);
  const sql = `
    SELECT
      COUNT(*) AS total_products,
      COALESCE(SUM(stock_quantity), 0) AS total_stock,
      COALESCE(ROUND(AVG(unit_price), 2), 0) AS average_price,
      COALESCE(ROUND(SUM(stock_quantity * unit_price), 2), 0) AS total_inventory_value
    FROM products${whereSql}
  `;
  const [rows] = await pool.execute(sql, params);
  const summary = rows[0] || {};
  res.json({
    total_products: Number(summary.total_products || 0),
    total_stock: Number(summary.total_stock || 0),
    average_price: Number(summary.average_price || 0),
    total_inventory_value: Number(summary.total_inventory_value || 0),
  });
});
```

```
  app.get('/api/products', async (req, res) => {
    const { whereSql, params } = buildFilterClause(req.query);
    const sql = `
      SELECT product_id, product_name, category, stock_quantity, unit_price, managed_by
      FROM products${whereSql}
      ORDER BY product_id ASC
    `;
    const [rows] = await pool.execute(sql, params);
    res.json(
      rows.map((row) => ({
        ...row,
        stock_quantity: Number(row.stock_quantity),
        unit_price: Number(row.unit_price),
      }))
    );
  });

  app.post('/api/products', async (req, res) => {
    const { error, value } = validateProduct(req.body);
    if (error) return res.status(400).json({ error });

    const canonicalCategory = await resolveCategoryName(value.category);
    if (!canonicalCategory) {
      return res.status(400).json({
        error: 'Selected category does not exist. Please add the category first.',
      });
    }
    value.category = canonicalCategory;

    const currentCount = await resequenceProducts(pool);
    const nextId = currentCount + 1;

    const sql = `
      INSERT INTO products (product_id, product_name, category, stock_quantity, unit_price,
managed_by)
      VALUES (?, ?, ?, ?, ?, ?)
    `;
    await pool.execute(sql, [
      nextId,
      value.product_name,
      value.category,
      value.stock_quantity,
      value.unit_price,
      value.managed_by,
    ]);

    if (syncSqlFile) await syncDatabaseSqlFile(pool);
    res.status(201).json({ product_id: nextId, ...value });
  });

  app.put('/api/products/:id', async (req, res) => {
    const id = parseId(req.params.id);
    if (!id) return res.status(400).json({ error: 'Invalid product ID.' });

    const { error, value } = validateProduct(req.body);
    if (error) return res.status(400).json({ error });

    const canonicalCategory = await resolveCategoryName(value.category);
    if (!canonicalCategory) {
      return res.status(400).json({
        error: 'Selected category does not exist. Please add the category first.',
      });
    }
    value.category = canonicalCategory;

    const sql = `
      UPDATE products
```

```
      SET product_name = ?, category = ?, stock_quantity = ?, unit_price = ?, managed_by = ?
      WHERE product_id = ?
    `;
    const [result] = await pool.execute(sql, [
      value.product_name,
      value.category,
      value.stock_quantity,
      value.unit_price,
      value.managed_by,
      id,
    ]);

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Product record not found.' });
    }
    if (syncSqlFile) await syncDatabaseSqlFile(pool);
    res.json({ product_id: id, ...value });
  });

  app.delete('/api/products/:id', async (req, res) => {
    const id = parseId(req.params.id);
    if (!id) return res.status(400).json({ error: 'Invalid product ID.' });

    const [result] = await pool.execute('DELETE FROM products WHERE product_id = ?', [id]);
    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Product record not found.' });
    }
    await resequenceProducts(pool);
    if (syncSqlFile) await syncDatabaseSqlFile(pool);
    res.json({ message: 'Product deleted successfully.', product_id: id });
  });

  app.use((err, _req, res, _next) => {
    console.error(err);
    res.status(500).json({ error: 'Database operation failed.' });
  });

  return app;
}

async function bootstrapDatabase({ forceSeed = false } = {}) {
  const { database, ...serverOnlyConfig } = dbConfig;
  const conn = await mysql.createConnection({ ...serverOnlyConfig, multipleStatements: true });
  try {
    const [existingProducts] = await conn
      .query(`SHOW TABLES FROM \`${database}\` LIKE 'products'`)
      .catch(() => [[]]);

    const fullSql = fs.readFileSync(path.join(__dirname, 'database.sql'), 'utf8');
    const seedIndex = fullSql.indexOf('SET FOREIGN_KEY_CHECKS');
    const schemaSql = seedIndex >= 0 ? fullSql.slice(0, seedIndex) : fullSql;
    const seedPart = seedIndex >= 0 ? fullSql.slice(seedIndex) : '';
    await conn.query(schemaSql);

    if (forceSeed || existingProducts.length === 0) {
      await conn.query(`USE \`${database}\`;\n${seedPart}`);
    }
  } finally {
    await conn.end();
  }
}

function createLocalFallbackPool() {
  const { DatabaseSync } = require('node:sqlite');
  const dbPath = path.join(os.tmpdir(), 'feni-uni-inventory-local.sqlite');
  const db = new DatabaseSync(dbPath);

  db.exec('PRAGMA foreign_keys = OFF; DROP TABLE IF EXISTS products; DROP TABLE IF EXISTS
```

```
categories;');
  db.exec('PRAGMA foreign_keys = ON;');
  db.exec(`
    CREATE TABLE IF NOT EXISTS categories (
      category_id INTEGER PRIMARY KEY,
      category_name TEXT NOT NULL UNIQUE
    );
    CREATE TABLE IF NOT EXISTS products (
      product_id INTEGER PRIMARY KEY,
      product_name TEXT NOT NULL,
      category TEXT NOT NULL,
      stock_quantity INTEGER NOT NULL CHECK (stock_quantity >= 0),
      unit_price REAL NOT NULL CHECK (unit_price >= 0.00),
      managed_by TEXT NOT NULL,
      FOREIGN KEY (category) REFERENCES categories(category_name)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
    );
  `);

  const seedSql = fs.readFileSync(path.join(__dirname, 'database.sql'), 'utf8');
  const catInsert = seedSql.match(/INSERT INTO categories[\s\S]*?;/);
  const prodInsert = seedSql.match(/INSERT INTO products[\s\S]*?;/);

  if (catInsert) db.exec(catInsert[0]);
  if (prodInsert) db.exec(prodInsert[0]);

  return {
    async execute(sql, params = []) {
      const stmt = db.prepare(sql);
      if (sql.trim().toUpperCase().startsWith('SELECT')) {
        return [stmt.all(...params)];
      }
      const res = stmt.run(...params);
      return [{ insertId: Number(res.lastInsertRowid), affectedRows: Number(res.changes) }];
    },
  };
}

if (require.main === module) {
  const forceSeed = process.argv.includes('--seed');
  const port = Number(process.env.PORT || 3000);

  (async () => {
    let pool;
    try {
      await bootstrapDatabase({ forceSeed });
      if (forceSeed) process.exit(0);
      pool = mysql.createPool({
        ...dbConfig,
        waitForConnections: true,
        connectionLimit: 10,
        decimalNumbers: true,
      });
    } catch (_err) {
      pool = createLocalFallbackPool();
      if (forceSeed) process.exit(0);
    }

    await resequenceCategories(pool);
    await resequenceProducts(pool);
    await syncDatabaseSqlFile(pool);

    const app = createApp(pool, { syncSqlFile: true });
    app.listen(port, () => {
      console.log(`Server running at http://localhost:${port}`);
    });
  })();
```

```
}

module.exports = { createApp, validateProduct, validateCategory, buildFilterClause };
```

7.4 Frontend HTML (`public/index.html`)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Inventory / Product Management System</title>
  <link rel="icon" type="image/svg+xml" href="/favicon.svg?v=2">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <header class="page-header">
      <h1>Inventory / Product Management System</h1>
      <span class="db-badge">MySQL</span>
    </header>

    <div id="status-banner" class="status-banner"></div>

    <div class="dashboard-grid">
      <section class="summary-grid">
        <article class="summary-card">
          <div class="summary-label">Total Products</div>
          <p id="sum-products" class="summary-value">0</p>
        </article>

        <article class="summary-card">
          <div class="summary-label">Total Stock Units</div>
          <p id="sum-stock" class="summary-value">0</p>
        </article>

        <article class="summary-card">
          <div class="summary-label">Average Unit Price</div>
          <p id="avg-price" class="summary-value">৳0.00</p>
        </article>

        <article class="summary-card">
          <div class="summary-label">Total Stock Valuation</div>
          <p id="sum-valuation" class="summary-value">৳0.00</p>
        </article>
      </section>

      <main class="workspace">
        <div class="sidebar-stack">
          <section class="panel">
            <form id="add-product-form" novalidate>
              <h2 class="panel-title form-heading">Add New Product</h2>

              <div class="form-group">
                <label for="add-name">Product Name *</label>
                <span class="hint">Enter product name (max 100 chars)</span>
                <input
                  type="text"
                  id="add-name"
                  name="product_name"
                  required
                  maxlength="100"
                  placeholder="e.g., Digital Milk Analyzer"
                >
                <div class="error-msg">Please enter a product name.</div>
              </div>
```

```
              <div class="form-group">
                <label id="add-category-label" for="add-category">Category *</label>
                <select id="add-category" name="category" required>
                  <option value="" disabled selected>Select category...</option>
                </select>
                <div id="add-category-error" class="error-msg">Please choose a category.</div>
              </div>

              <div class="form-group">
                <label for="add-stock">Stock Quantity *</label>
                <span class="hint">Available units in stock</span>
                <input
                  type="number"
                  id="add-stock"
                  name="stock_quantity"
                  required
                  min="0"
                  step="1"
                  placeholder="e.g., 25"
                >
                <div class="error-msg">Enter a valid stock quantity.</div>
              </div>

              <div class="form-group">
                <label for="add-price">Unit Price (BDT ৳) *</label>
                <span class="hint">Price per unit in Taka</span>
                <input
                  type="number"
                  id="add-price"
                  name="unit_price"
                  required
                  min="0"
                  max="99999999.99"
                  step="0.01"
                  placeholder="e.g., 1450.00"
                >
                <div class="error-msg">Enter a valid price in BDT.</div>
              </div>

              <div class="form-group">
                <label for="add-manager">Managed By *</label>
                <span class="hint">Store manager name</span>
                <input
                  type="text"
                  id="add-manager"
                  name="managed_by"
                  required
                  maxlength="60"
                  placeholder="e.g., Rejowan Ferdous Khan"
                >
                <div class="error-msg">Please enter the manager's name.</div>
              </div>

              <button type="submit" id="add-submit-btn" class="btn-primary">Save Product
Record</button>
            </form>
          </section>

          <section class="panel">
            <div class="panel-header">
              <h2 class="panel-title">Manage Categories</h2>
              <span id="cat-count-badge" class="count-pill">0</span>
            </div>

            <form id="add-category-form" novalidate>
              <label for="new-category-name">Category Name *</label>
              <span class="hint">Add or remove product categories</span>
```

```
              <div class="cat-input-group">
                <input
                  type="text"
                  id="new-category-name"
                  name="category_name"
                  required
                  maxlength="50"
                  placeholder="e.g., Poultry Care"
                  autocomplete="off"
                >
                <button type="submit" id="add-cat-btn" class="btn-primary">Add</button>
              </div>
              <div id="new-cat-error" class="error-msg">Please enter a category name.</div>
            </form>

            <ul id="category-list" class="category-list"></ul>
          </section>
        </div>

        <section class="panel records-panel">
          <form id="filter-form" class="filter-bar">
            <div class="form-group">
              <label for="search-input">Search Inventory</label>
              <input
                type="search"
                id="search-input"
                name="search"
                placeholder="Search product..."
                autocomplete="off"
              >
            </div>

            <div class="form-group">
              <label id="category-filter-label" for="category-filter">Filter Category</label>
              <select id="category-filter" name="category">
                <option value="">All Categories</option>
              </select>
            </div>

            <button type="reset" id="reset-filter-btn" class="btn-secondary">Clear</button>
          </form>

          <div tabindex="0" role="region" aria-labelledby="inventory-table-caption" class="table-
wrapper">
            <table>
              <caption id="inventory-table-caption">Saved Inventory Records</caption>
              <thead>
                <tr>
                  <th class="col-id">ID</th>
                  <th class="col-name">Product Name</th>
                  <th class="col-center">Category</th>
                  <th class="col-center">Stock Qty</th>
                  <th class="col-center">Unit Price (৳)</th>
                  <th class="col-center">Managed By</th>
                  <th class="col-center">Actions</th>
                </tr>
              </thead>
              <tbody id="inventory-tbody">
                <tr>
                  <td colspan="7" class="empty-state">Loading inventory records...</td>
                </tr>
              </tbody>
            </table>
          </div>
        </section>
      </main>
    </div>
  </div>
```

```
  <dialog id="edit-dialog">
    <h2 id="edit-dialog-title">Update Product Record</h2>
    <form id="edit-product-form" novalidate>
      <input type="hidden" id="edit-id" name="product_id">

      <div class="form-group">
        <label for="edit-name">Product Name *</label>
        <input type="text" id="edit-name" name="product_name" required maxlength="100">
        <div class="error-msg">Please enter a product name.</div>
      </div>

      <div class="form-group">
        <label id="edit-category-label" for="edit-category">Category *</label>
        <select id="edit-category" name="category" required></select>
        <div id="edit-category-error" class="error-msg">Please choose a category.</div>
      </div>

      <div class="form-group">
        <label for="edit-stock">Stock Quantity *</label>
        <input type="number" id="edit-stock" name="stock_quantity" required min="0" step="1">
        <div class="error-msg">Enter a valid stock quantity.</div>
      </div>

      <div class="form-group">
        <label for="edit-price">Unit Price (BDT ৳) *</label>
        <input type="number" id="edit-price" name="unit_price" required min="0" max="99999999.99"
step="0.01">
        <div class="error-msg">Enter a valid price in BDT.</div>
      </div>

      <div class="form-group">
        <label for="edit-manager">Managed By *</label>
        <input type="text" id="edit-manager" name="managed_by" required maxlength="60">
        <div class="error-msg">Please enter the manager's name.</div>
      </div>

      <div class="dialog-actions">
        <button type="button" id="edit-cancel-btn" class="btn-secondary">Cancel</button>
        <button type="submit" id="edit-submit-btn" class="btn-primary">Update Changes</button>
      </div>
    </form>
  </dialog>

  <dialog id="delete-dialog">
    <h2 id="delete-dialog-title">Delete Inventory Record?</h2>
    <p id="delete-dialog-message">Are you sure you want to permanently remove this product?</p>
    <form method="dialog" class="dialog-actions">
      <button id="delete-cancel-btn" value="cancel" class="btn-secondary">Cancel</button>
      <button id="delete-confirm-btn" value="delete" class="btn-danger">Delete Permanently</button>
    </form>
  </dialog>

  <script src="script.js" defer></script>
</body>
</html>
```

7.5 Frontend Stylesheet (`public/style.css`)

```
:root {
  --bg: #f8fafc;
  --surface: #ffffff;
  --surface-muted: #f1f5f9;
  --border: #cbd5e1;
  --text: #0f172a;
  --text-muted: #475569;
```

```
  --primary: #0f766e;
  --primary-hover: #115e59;
  --danger: #b91c1c;
  --danger-bg: #fef2f2;
  --success: #15803d;
  --success-bg: #f0fdf4;
  --focus: #0b57d0;
  --radius: 10px;
  --shadow: 0 1px 3px rgba(15, 23, 42, 0.08);
}

*, *::before, *::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  font-size: 1rem;
  line-height: 1.5;
  color: var(--text);
  background: var(--bg);
}

.container {
  max-width: 1380px;
  margin: 0 auto;
  padding: 1.5rem 1.5rem 3rem;
}

.page-header {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
  padding-bottom: 1.25rem;
  margin-bottom: 1.25rem;
  border-bottom: 1px solid var(--border);
}

.page-header h1 {
  margin: 0;
  font-size: 1.6rem;
}

.db-badge {
  display: inline-flex;
  align-items: center;
  padding: 0.35rem 0.85rem;
  border-radius: 999px;
  background: #ccfbf1;
  color: #115e59;
  font-size: 0.85rem;
  font-weight: 600;
}

.status-banner {
  min-height: 0;
}

.status-banner[data-state="visible"] {
  margin-bottom: 1.25rem;
  padding: 0.75rem 1rem;
  border-radius: var(--radius);
  border: 1px solid var(--border);
  font-weight: 500;
}
```

```
.status-banner[data-tone="success"] {
  background: var(--success-bg);
  border-color: #86efac;
  color: var(--success);
}

.status-banner[data-tone="error"] {
  background: var(--danger-bg);
  border-color: #fca5a5;
  color: var(--danger);
}

.dashboard-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 1.25rem;
}

.summary-grid {
  grid-column: 1 / -1;
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 1.25rem;
}

.summary-card {
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 1.15rem 1.25rem;
  box-shadow: var(--shadow);
}

.summary-label {
  color: var(--text-muted);
  font-size: 0.875rem;
  font-weight: 600;
}

.summary-value {
  margin: 0.45rem 0 0;
  font-size: 1.65rem;
  font-weight: 700;
}

.workspace {
  grid-column: 1 / -1;
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 1.25rem;
  align-items: start;
}

.sidebar-stack {
  grid-column: span 1;
  display: grid;
  gap: 1.25rem;
  min-width: 0;
}

.records-panel {
  grid-column: span 3;
  min-width: 0;
}
```

```
@media (max-width: 1040px) {
  .dashboard-grid,
  .summary-grid,
  .workspace {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .sidebar-stack,
  .records-panel {
    grid-column: 1 / -1;
  }

  .sidebar-stack {
    grid-template-columns: repeat(2, minmax(0, 1fr));
    align-items: start;
  }
}

@media (max-width: 640px) {
  .container {
    padding-left: 1rem;
    padding-right: 1rem;
  }

  .dashboard-grid,
  .summary-grid,
  .workspace,
  .sidebar-stack {
    grid-template-columns: 1fr;
  }
}

.panel {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 1.25rem;
  box-shadow: var(--shadow);
}

.panel-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 0.5rem;
  margin-bottom: 0.9rem;
}

.panel-title {
  margin: 0;
  font-size: 1.15rem;
  font-weight: 700;
}

.form-heading {
  margin-bottom: 1rem;
}

.count-pill {
  display: inline-flex;
  align-items: center;
  padding: 0.15rem 0.55rem;
  border-radius: 999px;
  background: var(--surface-muted);
  border: 1px solid var(--border);
  color: var(--text-muted);
  font-size: 0.78rem;
```

```
  font-weight: 600;
}

.form-group {
  margin-bottom: 1.1rem;
}

label {
  display: inline-block;
  font-weight: 600;
  font-size: 0.925rem;
  margin-bottom: 0.3rem;
}

.hint {
  display: block;
  color: var(--text-muted);
  font-size: 0.8rem;
  margin-bottom: 0.3rem;
}

input, select {
  width: 100%;
  font-family: inherit;
  font-size: 1rem;
  padding: 0.65rem 0.75rem;
  min-height: 48px;
  color: var(--text);
  background-color: var(--surface);
  border: 1px solid #94a3b8;
  border-radius: 8px;
}

input[type="search"] {
  appearance: none;
}

#search-input {
  padding-left: 2.35rem;
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16'
height='16' viewBox='0 0 24 24' fill='none' stroke='%2364748b' stroke-width='2.2' stroke-
linecap='round' stroke-linejoin='round'%3E%3Ccircle cx='11' cy='11' r='8'/%3E%3Cpath d='m21 21-4.3-
4.3'/%3E%3C/svg%3E");
  background-repeat: no-repeat;
  background-position: left 0.8rem center;
  background-size: 1rem;
}

.sr-only-select {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  border: 0;
  pointer-events: none;
}

.custom-select {
  position: relative;
  width: 100%;
}

.custom-select-btn {
  width: 100%;
  display: flex;
```

```
  align-items: center;
  justify-content: space-between;
  gap: 0.5rem;
  padding: 0.65rem 0.75rem;
  min-height: 48px;
  font-size: 1rem;
  font-weight: 400;
  text-align: left;
  color: var(--text);
  background: var(--surface);
  border: 1px solid #94a3b8;
  border-radius: 8px;
}

.custom-select-btn:hover {
  border-color: #64748b;
}

.custom-select-btn.is-placeholder {
  color: var(--text-muted);
}

.custom-select-label {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.custom-select-chevron {
  flex-shrink: 0;
  color: var(--text-muted);
  transition: transform 0.15s ease;
}

.custom-select-btn[aria-expanded="true"] .custom-select-chevron {
  transform: rotate(180deg);
}

.custom-select-menu {
  position: absolute;
  top: calc(100% + 0.35rem);
  left: 0;
  right: 0;
  z-index: 30;
  margin: 0;
  padding: 0.35rem;
  list-style: none;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  box-shadow: 0 12px 28px rgba(15, 23, 42, 0.14);
  max-height: 300px;
  overflow-y: auto;
}

.custom-select-menu[hidden] {
  display: none;
}

.custom-select-option {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.5rem;
  padding: 0.55rem 0.7rem;
  border-radius: 6px;
  font-size: 0.94rem;
  color: var(--text);
```

```
  cursor: pointer;
}

.custom-select-option:hover,
.custom-select-option.is-active {
  background: var(--surface-muted);
}

.custom-select-option[aria-selected="true"] {
  background: #f0fdfa;
  color: var(--primary-hover);
  font-weight: 600;
}

.custom-select-check {
  color: var(--primary);
  font-weight: 700;
  font-size: 0.88rem;
}

input:focus-visible,
select:focus-visible,
button:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 2px;
}

.error-msg {
  display: none;
  color: var(--danger);
  font-size: 0.84rem;
  font-weight: 500;
  margin-top: 0.3rem;
}

input.is-invalid,
select.is-invalid,
.custom-select.is-invalid .custom-select-btn {
  border-color: var(--danger);
  background-color: var(--danger-bg);
}

input.is-invalid + .error-msg,
select.is-invalid + .error-msg,
.cat-input-group.is-invalid + .error-msg {
  display: block;
}

.cat-input-group {
  display: flex;
  gap: 0.5rem;
  align-items: stretch;
}

.cat-input-group input {
  min-width: 0;
  flex: 1 1 auto;
}

.cat-input-group .btn-primary {
  width: auto;
  padding-left: 1.05rem;
  padding-right: 1.05rem;
  white-space: nowrap;
}

.category-list {
  list-style: none;
```

```
  margin: 0.9rem 0 0;
  padding: 0.4rem;
  display: grid;
  gap: 0.4rem;
  max-height: 340px;
  overflow-y: auto;
  background: var(--surface-muted);
  border: 1px solid var(--border);
  border-radius: 8px;
}

.category-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.5rem;
  padding: 0.45rem 0.6rem;
  background: var(--surface);
  border: 1px solid #e2e8f0;
  border-radius: 6px;
}

.category-meta {
  display: flex;
  align-items: center;
  gap: 0.45rem;
  min-width: 0;
}

.category-name {
  font-size: 0.86rem;
  font-weight: 600;
  color: var(--text);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.category-count {
  flex-shrink: 0;
  padding: 0.1rem 0.45rem;
  border-radius: 999px;
  background: #e2e8f0;
  color: var(--text-muted);
  font-size: 0.74rem;
  font-weight: 600;
}

.btn-cat-delete {
  flex-shrink: 0;
  min-height: 30px;
  padding: 0.2rem 0.55rem;
  font-size: 0.78rem;
  font-weight: 600;
  color: var(--danger);
  background: var(--danger-bg);
  border: 1px solid #fecaca;
  border-radius: 6px;
}

.btn-cat-delete:hover {
  background: #fee2e2;
  border-color: #fca5a5;
}

button {
  font-family: inherit;
  font-size: 0.95rem;
```

```
  font-weight: 600;
  min-height: 48px;
  padding: 0.6rem 1rem;
  border-radius: 8px;
  border: 1px solid transparent;
  cursor: pointer;
  transition: background-color 0.15s ease;
}

.btn-primary {
  width: 100%;
  background: var(--primary);
  color: #ffffff;
}

.btn-primary:hover:not(:disabled) {
  background: var(--primary-hover);
}

.btn-primary:disabled {
  opacity: 0.65;
  cursor: wait;
}

.btn-secondary {
  background: var(--surface-muted);
  color: var(--text);
  border-color: var(--border);
}

.btn-secondary:hover {
  background: #e2e8f0;
}

.btn-danger {
  background: var(--danger);
  color: #ffffff;
}

.btn-danger:hover {
  background: #991b1b;
}

.btn-sm {
  min-height: 34px;
  padding: 0.3rem 0.6rem;
  font-size: 0.82rem;
}

.filter-bar {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 220px auto;
  gap: 0.75rem;
  align-items: end;
  margin-bottom: 1rem;
}

@media (max-width: 680px) {
  .filter-bar {
    grid-template-columns: 1fr;
  }
}

.filter-bar .form-group {
  margin-bottom: 0;
}

.table-wrapper {
```

```
  overflow: auto;
  max-width: 100%;
  max-height: 1120px;
  border: 1px solid var(--border);
  border-radius: 8px;
}

table {
  width: 100%;
  border-collapse: separate;
  border-spacing: 0;
}

caption {
  text-align: left;
  padding: 0.75rem 1rem;
  font-weight: 700;
  background: var(--surface-muted);
  border-bottom: 1px solid var(--border);
}

thead {
  position: sticky;
  top: 0;
  z-index: 2;
}

thead th {
  background: #e2e8f0;
  color: var(--text);
  font-size: 0.82rem;
  text-align: left;
  padding: 0.65rem 0.4rem;
  border-bottom: 1px solid var(--border);
  white-space: nowrap;
}

tbody th,
tbody td {
  padding: 0.6rem 0.4rem;
  border-bottom: 1px solid #e2e8f0;
  vertical-align: middle;
  font-size: 0.86rem;
}

tbody th {
  text-align: left;
  font-weight: 600;
}

thead th.col-id,
tbody td.col-id {
  width: 48px;
  text-align: center;
  padding-left: 0.4rem;
  padding-right: 0.4rem;
}

thead th.col-name,
tbody th.col-name {
  text-align: left;
  padding-left: 0.45rem;
  padding-right: 0.5rem;
  white-space: nowrap;
}

thead th.col-center,
tbody td.col-center {
```

```
  text-align: center;
}

tbody tr:hover th,
tbody tr:hover td {
  background: #f8fafc;
}

.id-box {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 30px;
  height: 28px;
  font-size: 0.8rem;
  font-weight: 700;
  color: var(--primary-hover);
  background: #f0fdfa;
  border: 1px solid #99f6e4;
  border-radius: 6px;
}

.cat-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-width: 126px;
  padding: 0.22rem 0.55rem;
  border-radius: 999px;
  font-size: 0.77rem;
  font-weight: 600;
  background: #f1f5f9;
  border: 1px solid var(--border);
  white-space: nowrap;
}

.low-stock {
  color: var(--danger);
  font-weight: 700;
}

.num-cell {
  white-space: nowrap;
}

.manager-cell {
  white-space: nowrap;
}

.row-actions {
  display: flex;
  justify-content: center;
  gap: 0.4rem;
  flex-wrap: nowrap;
}

.empty-state {
  text-align: center;
  padding: 2rem 1rem;
  color: var(--text-muted);
}

dialog {
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 1.5rem;
  width: min(480px, calc(100vw - 2rem));
  box-shadow: 0 20px 35px rgba(15, 23, 42, 0.2);
```

```
  color: var(--text);
  background: var(--surface);
}

dialog::backdrop {
  background-color: rgba(15, 23, 42, 0.55);
}

dialog h2 {
  margin: 0 0 1rem;
  font-size: 1.25rem;
}

.dialog-actions {
  display: flex;
  justify-content: flex-end;
  gap: 0.75rem;
  margin-top: 1.25rem;
}

.dialog-actions .btn-primary {
  width: auto;
}
```

7.6 Frontend JavaScript (`public/script.js`)

```
const bdtFormat = new Intl.NumberFormat('en-IN', {
  minimumFractionDigits: 2,
  maximumFractionDigits: 2,
});
const intFormat = new Intl.NumberFormat('en-IN');
const formatBDT = (amount) => `৳${bdtFormat.format(Number(amount || 0))}`;

const statusBanner = document.getElementById('status-banner');
const sumProductsEl = document.getElementById('sum-products');
const sumStockEl = document.getElementById('sum-stock');
const avgPriceEl = document.getElementById('avg-price');
const sumValuationEl = document.getElementById('sum-valuation');

const addForm = document.getElementById('add-product-form');
const addCategorySelect = document.getElementById('add-category');
const addSubmitBtn = document.getElementById('add-submit-btn');
const filterForm = document.getElementById('filter-form');
const searchInput = document.getElementById('search-input');
const categoryFilter = document.getElementById('category-filter');
const tbody = document.getElementById('inventory-tbody');
const tableCaption = document.getElementById('inventory-table-caption');

const editDialog = document.getElementById('edit-dialog');
const editForm = document.getElementById('edit-product-form');
const editCategorySelect = document.getElementById('edit-category');
const editCancelBtn = document.getElementById('edit-cancel-btn');
const editSubmitBtn = document.getElementById('edit-submit-btn');

const addCategoryForm = document.getElementById('add-category-form');
const newCategoryInput = document.getElementById('new-category-name');
const addCatBtn = document.getElementById('add-cat-btn');
const newCatError = document.getElementById('new-cat-error');
const catCountBadge = document.getElementById('cat-count-badge');
const categoryListEl = document.getElementById('category-list');

const deleteDialog = document.getElementById('delete-dialog');
const deleteDialogTitle = document.getElementById('delete-dialog-title');
const deleteMessage = document.getElementById('delete-dialog-message');
const deleteCancelBtn = document.getElementById('delete-cancel-btn');
const deleteConfirmBtn = document.getElementById('delete-confirm-btn');
```

```
const currentRecords = new Map();
const customSelectRegistry = new Map();
let categoriesData = [];
let categoryList = [];
let pendingDelete = null;
let bannerTimer = null;
let activeCustomClose = null;

function showBanner(message, tone = 'success') {
  clearTimeout(bannerTimer);
  statusBanner.textContent = message;
  statusBanner.dataset.state = 'visible';
  statusBanner.dataset.tone = tone;
  if (tone === 'success') {
    bannerTimer = setTimeout(() => {
      statusBanner.textContent = '';
      delete statusBanner.dataset.state;
      delete statusBanner.dataset.tone;
    }, 4500);
  }
}

function refreshCustomSelect(selectEl) {
  const sync = customSelectRegistry.get(selectEl);
  if (sync) sync();
}

function setupCustomSelect(select) {
  const wrapper = document.createElement('div');
  wrapper.className = 'custom-select';

  const btn = document.createElement('button');
  btn.type = 'button';
  btn.className = 'custom-select-btn';
  btn.setAttribute('aria-expanded', 'false');

  const labelSpan = document.createElement('span');
  labelSpan.className = 'custom-select-label';

  const chevron = document.createElementNS('http://www.w3.org/2000/svg', 'svg');
  chevron.setAttribute('width', '16');
  chevron.setAttribute('height', '16');
  chevron.setAttribute('viewBox', '0 0 24 24');
  chevron.setAttribute('fill', 'none');
  chevron.setAttribute('stroke', 'currentColor');
  chevron.setAttribute('stroke-width', '2.25');
  chevron.setAttribute('stroke-linecap', 'round');
  chevron.setAttribute('stroke-linejoin', 'round');
  chevron.classList.add('custom-select-chevron');
  const path = document.createElementNS('http://www.w3.org/2000/svg', 'path');
  path.setAttribute('d', 'm6 9 6 6 6-6');
  chevron.appendChild(path);

  btn.append(labelSpan, chevron);

  const menu = document.createElement('ul');
  menu.className = 'custom-select-menu';
  menu.hidden = true;

  wrapper.append(btn, menu);
  select.before(wrapper);
  select.classList.add('sr-only-select');
  select.tabIndex = -1;

  const sync = () => {
    const chosen = select.selectedOptions[0] || select.options[0];
    labelSpan.textContent = chosen ? chosen.textContent : '';
```

```
    btn.classList.toggle('is-placeholder', Boolean(select.required && !select.value));
    wrapper.classList.toggle('is-invalid', select.classList.contains('is-invalid'));

    menu.replaceChildren();
    for (const opt of select.options) {
      if (opt.disabled) continue;
      const li = document.createElement('li');
      li.className = 'custom-select-option';
      li.dataset.value = opt.value;
      const isSelected = opt.selected && (!select.required || Boolean(opt.value));
      li.setAttribute('aria-selected', String(isSelected));

      const txt = document.createElement('span');
      txt.textContent = opt.textContent;
      li.appendChild(txt);

      if (isSelected) {
        const chk = document.createElement('span');
        chk.className = 'custom-select-check';
        chk.textContent = '✓';
        li.appendChild(chk);
      }
      menu.appendChild(li);
    }
  };

  const closeMenu = () => {
    if (menu.hidden) return;
    menu.hidden = true;
    btn.setAttribute('aria-expanded', 'false');
    if (activeCustomClose === closeMenu) activeCustomClose = null;
  };

  const openMenu = () => {
    if (activeCustomClose && activeCustomClose !== closeMenu) activeCustomClose();
    sync();
    menu.hidden = false;
    btn.setAttribute('aria-expanded', 'true');
    activeCustomClose = closeMenu;
  };

  btn.addEventListener('click', () => {
    if (menu.hidden) openMenu();
    else closeMenu();
  });

  menu.addEventListener('click', (e) => {
    const li = e.target.closest('.custom-select-option');
    if (!li) return;
    select.value = li.dataset.value;
    select.classList.remove('is-invalid');
    sync();
    select.dispatchEvent(new Event('change', { bubbles: true }));
    closeMenu();
  });

  customSelectRegistry.set(select, sync);
  sync();
}

for (const select of [addCategorySelect, categoryFilter, editCategorySelect]) {
  setupCustomSelect(select);
}

document.addEventListener('click', (e) => {
  if (activeCustomClose && !e.target.closest('.custom-select')) {
    activeCustomClose();
  }
```

```
});

function validateFormFields(form) {
  let valid = true;
  for (const el of form.querySelectorAll('input, select')) {
    const fieldValid = el.checkValidity();
    el.classList.toggle('is-invalid', !fieldValid);
    if (el.parentElement && el.parentElement.classList.contains('cat-input-group')) {
      el.parentElement.classList.toggle('is-invalid', !fieldValid);
    }
    if (el.tagName === 'SELECT') refreshCustomSelect(el);
    if (!fieldValid) valid = false;
  }
  return valid;
}

function clearValidationState(form) {
  for (const el of form.querySelectorAll('input, select, .cat-input-group')) {
    el.classList.remove('is-invalid');
    if (el.tagName === 'SELECT') refreshCustomSelect(el);
  }
}

document.addEventListener('input', (e) => {
  if (e.target.matches && e.target.matches('input, select') && e.target.checkValidity()) {
    e.target.classList.remove('is-invalid');
    if (e.target.parentElement && e.target.parentElement.classList.contains('cat-input-group')) {
      e.target.parentElement.classList.remove('is-invalid');
    }
  }
});

function syncCategorySelects(names) {
  categoryList = names;
  const prevAdd = addCategorySelect.value;
  const prevFilter = categoryFilter.value;
  const prevEdit = editCategorySelect.value;

  addCategorySelect.replaceChildren();
  const addPlaceholder = document.createElement('option');
  addPlaceholder.value = '';
  addPlaceholder.disabled = true;
  addPlaceholder.defaultSelected = true;
  addPlaceholder.textContent = 'Select category...';
  addCategorySelect.appendChild(addPlaceholder);
  for (const name of names) {
    const opt = document.createElement('option');
    opt.value = name;
    opt.textContent = name;
    addCategorySelect.appendChild(opt);
  }
  addCategorySelect.value = names.includes(prevAdd) ? prevAdd : '';

  categoryFilter.replaceChildren();
  const allOpt = document.createElement('option');
  allOpt.value = '';
  allOpt.defaultSelected = true;
  allOpt.textContent = 'All Categories';
  categoryFilter.appendChild(allOpt);
  for (const name of names) {
    const opt = document.createElement('option');
    opt.value = name;
    opt.textContent = name;
    categoryFilter.appendChild(opt);
  }
  categoryFilter.value = names.includes(prevFilter) ? prevFilter : '';

  editCategorySelect.replaceChildren();
```

```
  for (const name of names) {
    const opt = document.createElement('option');
    opt.value = name;
    opt.textContent = name;
    editCategorySelect.appendChild(opt);
  }
  if (names.includes(prevEdit)) editCategorySelect.value = prevEdit;

  refreshCustomSelect(addCategorySelect);
  refreshCustomSelect(categoryFilter);
  refreshCustomSelect(editCategorySelect);
}

function renderCategoryList(categories) {
  catCountBadge.textContent = `${intFormat.format(categories.length)} total`;
  categoryListEl.replaceChildren();

  if (categories.length === 0) {
    const emptyLi = document.createElement('li');
    emptyLi.className = 'category-item';
    emptyLi.textContent = 'No categories saved yet.';
    categoryListEl.appendChild(emptyLi);
    return;
  }

  for (const cat of categories) {
    const count = Number(cat.product_count || 0);
    const li = document.createElement('li');
    li.className = 'category-item';

    const meta = document.createElement('div');
    meta.className = 'category-meta';

    const nameSpan = document.createElement('span');
    nameSpan.className = 'category-name';
    nameSpan.textContent = cat.category_name;

    const countSpan = document.createElement('span');
    countSpan.className = 'category-count';
    countSpan.textContent = intFormat.format(count);

    meta.append(nameSpan, countSpan);

    const delBtn = document.createElement('button');
    delBtn.type = 'button';
    delBtn.className = 'btn-cat-delete';
    delBtn.dataset.deleteCatId = String(cat.category_id);
    delBtn.textContent = 'Delete';

    li.append(meta, delBtn);
    categoryListEl.appendChild(li);
  }
}

async function loadCategories() {
  try {
    const res = await fetch('/api/categories');
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Failed to load categories.');
    categoriesData = data;
    syncCategorySelects(data.map((c) => c.category_name));
    renderCategoryList(data);
  } catch (err) {
    showBanner(err.message, 'error');
  }
}

newCategoryInput.addEventListener('input', () => {
```

```
  newCatError.textContent = 'Please enter a category name.';
});

addCategoryForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const category_name = newCategoryInput.value.replace(/\s+/g, ' ').trim();
  if (!category_name) {
    newCategoryInput.value = '';
    newCatError.textContent = 'Please enter a category name.';
    validateFormFields(addCategoryForm);
    return;
  }

  addCatBtn.disabled = true;
  try {
    const res = await fetch('/api/categories', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ category_name }),
    });
    const data = await res.json();
    if (!res.ok) {
      newCatError.textContent = data.error || 'Could not add category.';
      newCategoryInput.classList.add('is-invalid');
      newCategoryInput.parentElement.classList.add('is-invalid');
      newCategoryInput.focus();
      return;
    }

    addCategoryForm.reset();
    newCatError.textContent = 'Please enter a category name.';
    clearValidationState(addCategoryForm);
    await loadCategories();

    if (!addCategorySelect.value) {
      addCategorySelect.value = data.category_name;
      addCategorySelect.classList.remove('is-invalid');
      refreshCustomSelect(addCategorySelect);
    }

    showBanner(`Added category "${data.category_name}".`);
  } catch (err) {
    showBanner(err.message, 'error');
  } finally {
    addCatBtn.disabled = false;
  }
});

categoryListEl.addEventListener('click', (e) => {
  const delBtn = e.target.closest('button[data-delete-cat-id]');
  if (!delBtn) return;
  const catId = Number(delBtn.dataset.deleteCatId);
  const cat = categoriesData.find((c) => Number(c.category_id) === catId);
  if (!cat) return;

  const count = Number(cat.product_count || 0);
  if (count > 0) {
    pendingDelete = null;
    deleteDialogTitle.textContent = 'Category In Use';
    deleteMessage.textContent = `Cannot delete "${cat.category_name}" because ${count === 1 ? '1
product is' : `${intFormat.format(count)} products are`} still assigned to it. Reassign or delete
those products first.`;
    deleteConfirmBtn.hidden = true;
    deleteCancelBtn.textContent = 'Close';
    deleteDialog.showModal();
    return;
  }
```

```
  pendingDelete = { type: 'category', id: cat.category_id, name: cat.category_name };
  deleteDialogTitle.textContent = 'Delete Category?';
  deleteMessage.textContent = `Permanently remove the "${cat.category_name}" category from the
database? This action cannot be undone.`;
  deleteConfirmBtn.hidden = false;
  deleteConfirmBtn.textContent = 'Delete Category';
  deleteCancelBtn.textContent = 'Cancel';
  deleteDialog.showModal();
});

function buildQueryString() {
  const params = new URLSearchParams();
  const q = searchInput.value.trim();
  const cat = categoryFilter.value.trim();
  if (q) params.set('search', q);
  if (cat) params.set('category', cat);
  const qs = params.toString();
  return qs ? `?${qs}` : '';
}

function renderRows(products) {
  currentRecords.clear();
  tbody.replaceChildren();

  const countLabel = products.length === 1 ? '1 item' : `${products.length} items`;
  tableCaption.textContent = `Saved Inventory Records (${countLabel})`;

  if (products.length === 0) {
    const tr = document.createElement('tr');
    const td = document.createElement('td');
    td.colSpan = 7;
    td.className = 'empty-state';
    td.textContent = 'No matching inventory records found.';
    tr.appendChild(td);
    tbody.appendChild(tr);
    return;
  }

  for (const item of products) {
    currentRecords.set(item.product_id, item);
    const tr = document.createElement('tr');

    const tdId = document.createElement('td');
    tdId.className = 'col-id';
    const idBadge = document.createElement('span');
    idBadge.className = 'id-box';
    idBadge.textContent = String(item.product_id);
    tdId.appendChild(idBadge);

    const thName = document.createElement('th');
    thName.scope = 'row';
    thName.className = 'col-name';
    thName.textContent = item.product_name;

    const tdCategory = document.createElement('td');
    tdCategory.className = 'col-center';
    const catSpan = document.createElement('span');
    catSpan.className = 'cat-badge';
    catSpan.textContent = item.category;
    tdCategory.appendChild(catSpan);

    const tdStock = document.createElement('td');
    tdStock.className = 'col-center num-cell';
    if (item.stock_quantity <= 10) {
      tdStock.classList.add('low-stock');
      tdStock.textContent = `${intFormat.format(item.stock_quantity)} (Low)`;
    } else {
      tdStock.textContent = intFormat.format(item.stock_quantity);
```

```
    }

    const tdPrice = document.createElement('td');
    tdPrice.className = 'col-center num-cell';
    tdPrice.textContent = formatBDT(item.unit_price);

    const tdManager = document.createElement('td');
    tdManager.className = 'col-center manager-cell';
    tdManager.textContent = item.managed_by;

    const tdActions = document.createElement('td');
    tdActions.className = 'col-center';
    const actionWrap = document.createElement('div');
    actionWrap.className = 'row-actions';

    const editBtn = document.createElement('button');
    editBtn.type = 'button';
    editBtn.className = 'btn-secondary btn-sm';
    editBtn.dataset.action = 'edit';
    editBtn.dataset.id = String(item.product_id);
    editBtn.textContent = 'Edit';

    const delBtn = document.createElement('button');
    delBtn.type = 'button';
    delBtn.className = 'btn-danger btn-sm';
    delBtn.dataset.action = 'delete';
    delBtn.dataset.id = String(item.product_id);
    delBtn.textContent = 'Delete';

    actionWrap.append(editBtn, delBtn);
    tdActions.appendChild(actionWrap);

    tr.append(tdId, thName, tdCategory, tdStock, tdPrice, tdManager, tdActions);
    tbody.appendChild(tr);
  }
}

async function loadInventory() {
  const qs = buildQueryString();
  try {
    const [productsRes, summaryRes] = await Promise.all([
      fetch(`/api/products${qs}`),
      fetch(`/api/products/summary${qs}`),
    ]);

    const productsData = await productsRes.json();
    const summaryData = await summaryRes.json();

    if (!productsRes.ok) {
      throw new Error(productsData.error || 'Failed to load inventory.');
    }

    renderRows(productsData);
    sumProductsEl.textContent = intFormat.format(summaryData.total_products);
    sumStockEl.textContent = `${intFormat.format(summaryData.total_stock)} units`;
    avgPriceEl.textContent = formatBDT(summaryData.average_price);
    sumValuationEl.textContent = formatBDT(summaryData.total_inventory_value);
  } catch (err) {
    showBanner(err.message, 'error');
  }
}

function readFormPayload(form) {
  const fd = new FormData(form);
  return {
    product_name: String(fd.get('product_name') || '').trim(),
    category: String(fd.get('category') || '').trim(),
    stock_quantity: Number(fd.get('stock_quantity')),
```

```
    unit_price: Number(fd.get('unit_price')),
    managed_by: String(fd.get('managed_by') || '').trim(),
  };
}

addForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  if (!validateFormFields(addForm)) return;

  addSubmitBtn.disabled = true;
  try {
    const res = await fetch('/api/products', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(readFormPayload(addForm)),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Could not add product.');

    addForm.reset();
    clearValidationState(addForm);
    refreshCustomSelect(addCategorySelect);
    showBanner(`Added "${data.product_name}" (#${data.product_id}) to inventory.`);
    await Promise.all([loadInventory(), loadCategories()]);
  } catch (err) {
    showBanner(err.message, 'error');
  } finally {
    addSubmitBtn.disabled = false;
  }
});

let searchDebounce = null;
filterForm.addEventListener('submit', (e) => {
  e.preventDefault();
  loadInventory();
});

searchInput.addEventListener('input', () => {
  clearTimeout(searchDebounce);
  searchDebounce = setTimeout(loadInventory, 160);
});

categoryFilter.addEventListener('change', loadInventory);

filterForm.addEventListener('reset', () => {
  setTimeout(() => {
    refreshCustomSelect(categoryFilter);
    loadInventory();
  }, 0);
});

tbody.addEventListener('click', (e) => {
  const btn = e.target.closest('button[data-action]');
  if (!btn) return;
  const id = Number(btn.dataset.id);
  const record = currentRecords.get(id);
  if (!record) return;

  if (btn.dataset.action === 'edit') {
    clearValidationState(editForm);
    if (!categoryList.includes(record.category)) {
      syncCategorySelects([...categoryList, record.category].sort());
    }
    document.getElementById('edit-id').value = String(record.product_id);
    document.getElementById('edit-name').value = record.product_name;
    editCategorySelect.value = record.category;
    refreshCustomSelect(editCategorySelect);
    document.getElementById('edit-stock').value = String(record.stock_quantity);
```

```
    document.getElementById('edit-price').value = Number(record.unit_price).toFixed(2);
    document.getElementById('edit-manager').value = record.managed_by;
    editDialog.showModal();
  } else if (btn.dataset.action === 'delete') {
    pendingDelete = { type: 'product', id: record.product_id, name: record.product_name };
    deleteDialogTitle.textContent = 'Delete Inventory Record?';
    deleteMessage.textContent = `Delete "#${record.product_id} ${record.product_name}" (managed by
${record.managed_by})? This action cannot be undone.`;
    deleteConfirmBtn.hidden = false;
    deleteConfirmBtn.textContent = 'Delete Permanently';
    deleteCancelBtn.textContent = 'Cancel';
    deleteDialog.showModal();
  }
});

editCancelBtn.addEventListener('click', () => editDialog.close());
editDialog.addEventListener('close', () => {
  if (activeCustomClose) activeCustomClose();
});

editForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  if (!validateFormFields(editForm)) return;

  const id = Number(document.getElementById('edit-id').value);
  editSubmitBtn.disabled = true;
  try {
    const res = await fetch(`/api/products/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(readFormPayload(editForm)),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Could not update product.');

    editDialog.close();
    showBanner(`Updated "#${data.product_id} ${data.product_name}" successfully.`);
    await Promise.all([loadInventory(), loadCategories()]);
  } catch (err) {
    showBanner(err.message, 'error');
  } finally {
    editSubmitBtn.disabled = false;
  }
});

deleteDialog.addEventListener('close', async () => {
  const action = deleteDialog.returnValue;
  deleteDialog.returnValue = '';
  const target = pendingDelete;
  pendingDelete = null;

  if (action !== 'delete' || !target) return;

  if (target.type === 'category') {
    try {
      const res = await fetch(`/api/categories/${target.id}`, { method: 'DELETE' });
      const data = await res.json();
      if (!res.ok) throw new Error(data.error || 'Could not delete category.');

      if (categoryFilter.value === target.name) categoryFilter.value = '';
      if (addCategorySelect.value === target.name) addCategorySelect.value = '';
      showBanner(`Deleted category "${target.name}".`);
      await Promise.all([loadCategories(), loadInventory()]);
    } catch (err) {
      showBanner(err.message, 'error');
    }
    return;
  }
```

```
  try {
    const res = await fetch(`/api/products/${target.id}`, { method: 'DELETE' });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Could not delete product.');

    showBanner(`Deleted product #${target.id} from inventory.`);
    await Promise.all([loadInventory(), loadCategories()]);
  } catch (err) {
    showBanner(err.message, 'error');
  }
});

Promise.all([loadCategories(), loadInventory()]);
```

8. Conclusion

This project implements a complete single-page Inventory / Product Management System using HTML5, CSS3, JavaScript, Node.js, Express, and MySQL. It satisfies all requirements of the Database Management System Lab by demonstrating relational schema design with primary and foreign keys, CRUD operations, search and category filtering with SQL `WHERE` conditions, and real-time summary calculations using SQL aggregate functions (`COUNT` , `SUM` , and `AVG`).